2026학년도 2학기 인문데이터 수량분석

제4강 탐색적 자료 분석 (2)

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 9월 14일 월요일

오늘의 목표

1. 조건에 맞는 행만 고르고, 필요한 열만 남기고, 원하는 순서로 정렬할 수 있다.
2. `group_by()`와 `summarize()`로 집단별 요약값을 계산할 수 있다.
3. `mutate()`로 새 열을 만들고 조건에 따라 값을 나눌 수 있다.
4. 여러 동사를 파이프로 이어 붙여 자료 손질부터 그림까지 한 번에 쓸 수 있다.

사전 준비

1. Posit Cloud의 `qah-2026` 프로젝트를 연다.
2. `hdr/data/`에 `acs_cbsa.csv`가 있는지 확인한다.
3. 콘솔에서 `library(tidyverse)`와 `library(ggrepel)`을 실행해 둔다.

1 **데이터 동사**
왜 자료를 손보는가
동사의 문법

2 행 고르기

3 열 고르기와 정렬

4 집단별 요약

5 새 열 만들기

6 마무리

3강에서 미뤄 둔 것들

그림이 아니라 자료의 문제였던 세 가지

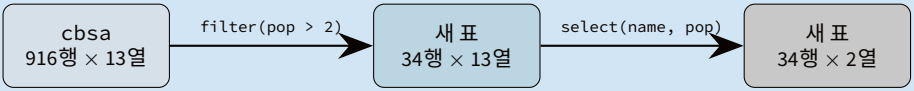
1. 934곳 가운데 **30곳**은 어떻게 골랐는가?
2. 막대가 왜 **알파벳 순**으로 늘어서는가?
3. 범례의 순서는 어떻게 바꾸는가?

셋 다 `ggplot2`로는 풀 수 없다. 그림에 넣기 **전에** 자료를 손봐야 한다.

엑셀로도 되는 일 아닌가?

- 정렬과 필터를 손으로 반복하면 언젠가는 같은 결과에 닿는다.
- ❗ 그러나 **무엇을 했는지가 남지 않는다.** 자료가 한 줄만 바뀌어도 처음부터 다시 해야 하고, 남이 그 과정을 재현할 수 없다.
- ➔ 코드는 결과가 아니라 **절차**를 기록한다. 논문의 각주가 하는 일과 같다.

데이터 동사(data verb)



원본은 그대로 916행

모든 동사가 지키는 세 가지

1. 첫 번째 인자는 **표**(tibble)이고, 결과도 **표**다.
2. 원본은 **절대 바뀌지 않는다**. 동사는 사본 위에서 일한다.
3. 그래서 파이프 |>로 얼마든지 이어 붙일 수 있다.

오늘의 일곱 동사

동사	하는 일	결과
`slice()`	번호로 행 고르기	행이 줄어든다
`filter()`	조건으로 행 고르기	행이 줄어든다
`select()`	열 고르기	열이 줄어든다
`arrange()`	행 정렬하기	순서만 바뀐다
`mutate()`	새 열 만들기	열이 늘어난다
`group_by()`	집단 표시하기	겉보기는 그대로
`summarize()`	여러 행을 한 줄로 접기	행이 확 줄어든다

- `dplyr` 패키지에는 40개가 넘는 동사가 있지만, 대부분은 이 일곱 개의 변형이다.
- 💡 `tidyverse`를 불러오면 `dplyr`도 함께 딸려 온다.

1 데이터 동사

2 **행 고르기**
번호와 조건
부분과 전체

3 열 고르기와 정렬

4 집단별 요약

5 새 열 만들기

6 마무리

오늘의 자료: 다시 cbsa

```
library(tidyverse)

cbsa <- read_csv(file.path("hdr", "data", "acs_cbsa.csv"))
cbsa <- filter(cbsa, quad != "0")   # 본토 밖 18곳을 뺀다
```

934곳에서 916곳으로

3강에서 미뤄 두었던 하와이, 알래스카, 푸에르토리코를 뺐다. !=는 "같지 않다"는 뜻이다.

3강과 다른 점

3강에서는 인구 상위 30곳만 그렸다. 오늘은 916곳 전부를 다룬다. 점이 뭉개져도 상관없다. 오늘의 관심은 그림이 아니라 표다.

오른쪽 끝에 ...이 붙은 출력은 화면에 다 들어가지 않아 **줄여 실은 것**이다. 실제 콘솔에는 열 이름이 모두 나온다.

번호로 고르기: slice()

```
cbsa |>
  slice(2, 5, 7)
```

```
# A tibble: 3 x 13
  name         geoid quad     lon   lat   pop density
  <chr>        <dbl> <chr>  <dbl> <dbl> <dbl>   <dbl>
1 Los Angeles  31080 W      -118.   34.2 13.2    1041.
2 Houston      26420 S       -95.4  29.8  7.05    317.
3 Philadelphia 37980 NE      -75.3  39.9  6.22    506.
# i 6 more variables: age_median <dbl>, ...
```

범위는 콜론으로

`slice(1:5)`는 1행부터 5행까지. `1:5`는 2강에서 본 그 표기다.

앞에서 몇 개

3강에서 쓴 `slice_head(cbsa, n = 30)`이 바로 `slice(1:30)`의 다른 이름이다.

원본은 바뀌지 않는다

앞의 코드를 실행한 뒤에도 cbsa는 여전히 916행이다

결과를 남기려면 **이름을 붙여 저장**해야 한다.

```
cbsa_first_five <- cbsa |>
  slice(1, 2, 3, 4, 5)
```

화살표와 파이프

|>는 표를 다음 함수로 **넘기고**, <-는 결과에 **이름을 붙인다**. 하는 일이 전혀 다르다.

덮어쓰기는 신중하게

`cbsa <- cbsa |> slice(1:5)`라고 쓰면 원본이 사라진다. 되돌리려면 read_csv부터 다시 해야 한다.

💡 탐색 중에는 저장하지 말고 그냥 실행해 보는 편이 낫다. 원본은 안전하다.

조건으로 고르기: filter()

```
cbsa |>
  filter(pop > 5)    # 인구 500만 명이 넘는 대도시권
```

```
# A tibble: 9 x 13
  name          geoid quad      lon   lat   pop density
  <chr>         <dbl> <chr>   <dbl> <dbl> <dbl>   <dbl>
1 New York      35620 NE      -74.1  40.8 20.0    1051.
2 Los Angeles   31080 W      -118.   34.2 13.2    1041.
3 Chicago       16980 NC      -88.0  41.7  9.61    509.
4 Dallas        19100 S       -97.0  32.8  7.54    323.
5 Houston       26420 S       -95.4  29.8  7.05    317.
6 Washington    47900 S       -77.5  38.8  6.33    364.
7 Philadelphia  37980 NE      -75.3  39.9  6.22    506.
8 Miami         33100 S       -80.5  26.2  6.11    430.
9 Atlanta       12060 S       -84.4  33.7  6.03    263.
# i 6 more variables: age_median <dbl>, ...
```

- pop은 백만 명 단위라 5는 500만 명이다. 참인 행만 남는다.

조건을 쓰는 방법

쓰는 법	뜻	보기
>, >=, <, <=	크기 비교	`filter(pop >= 1)`
==	같다	`filter(quad == "NE")`
!=	같지 않다	`filter(quad != "O")`
between(x, a, b)	a 이상 b 이하	`filter(between(pop, 1, 2))`
%in% c(...)	이 중 하나	`filter(quad %in% c("NE", "S"))`

결과의 크기

between(pop, 1, 2)은 21곳, quad %in% c("NE", "S")은 469곳이 남는다.

가장 흔한 실수

"같다"는 ==로 쓴다. = 하나는 함수의 인자를 지정하거나 값을 대입할 때 쓴다.

조건을 여러 개 잇기

```
cbsa |>
  filter(pop > 1) |>
  filter(quad %in% c("NE"))    # 인구 100만 이상, 북동부
```

```
# A tibble: 8 x 13
  name  geoid quad    lon   lat   pop density age_median
  <chr> <dbl> <chr> <dbl> <dbl> <dbl>   <dbl>      <dbl>
1 New   35620 NE     -74.1  40.8 20.0    1051.       42.9
2 Phil  37980 NE     -75.3  39.9  6.22    506.       42.6
3 Bost  14460 NE     -71.1  42.6  4.91    518.       42.2
# i 5 more rows, and 5 more variables: ...
```

한 줄로 써도 된다

`filter(pop > 1, quad == "NE")`처럼 콤마로 나열하면 "그리고"로 이어진다.

순서에 유의

`filter` 둘 사이에서는 순서가 결과를 바꾸지 않는다. 그러나 `slice`나 `arrange`와 섞이면 달라진다.

고르는 일이 곧 해석이다

코퍼스를 만드는 일 = `filter`를 쓰는 일

"19세기 소설만", "여성 작가만", "초간본만". 연구의 첫 문장은 거의 언제나 **무엇을 넣고 무엇을 뺄지**에 대한 결정이다.

국내 자료로 옮겨 보면

조선왕조실록(국사편찬위원회)에서 특정 왕대의 기사만 골라내는 일, 그것이 `filter`다. 왕대별로 묶어 세는 일이 뒤에 배울 `group_by`와 `summarize`다.

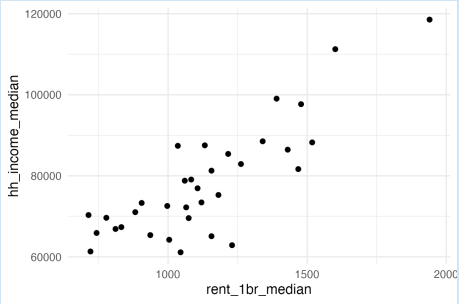
- 오늘 우리도 `quad != "0"`로 18곳을 뺐다. 하와이와 알래스카를 뺀 "미국"은 어떤 미국인가?

❗ 뺀 사실을 적지 않으면, 결론이 자료 때문인지 선택 때문인지 알 수 없다.

큰 도시만 따로 그려 보면

```
cbsa_large <- filter(cbsa, pop > 2)

cbsa_large |>
  ggplot(aes(rent_1br_median,
             hh_income_median)) +
    geom_point()
```



34곳이 남는다

인구 200만 명이 넘는 대도시권이다.
임대료가 높을수록 소득도 높아 보인다.

이 34곳이 나머지 882곳 속에서 **어디쯤**인지는 알 수 없다.

층마다 다른 자료: data =

```
cbsa |>
  ggplot(aes(rent_1br_median, hh_income_median)) +
    geom_point() +
    geom_point(data = cbsa_large)    # 이 층만 다른 표를 쓴다
```

층은 각자 다른 표를 쓸 수 있다

ggplot()에 넘긴 표가 기본값이고, 각 기하 객체에서 data =로 덮어쓸 수 있다. 첫 층은 916곳 전부, 둘째 층은 34곳만 그린다.

✗ 그런데 결과는 하나도 달라 보이지 않는다. 같은 검은 점이 같은 자리에 겹쳐 그려질 뿐이기 때문이다.

➔ 두 층을 **다른 색**으로 칠해야 비로소 보인다.

강조하기

```
library(ggrepel)

cbsa |>
  ggplot(aes(rent_1br_median, hh_income_median)) +
    geom_point(color = "grey85") +
    geom_point(color = "navy", data = cbsa_large) +
    geom_text_repel(
      aes(label = name), color = "navy", data = cbsa_large, size = 3
    )
```

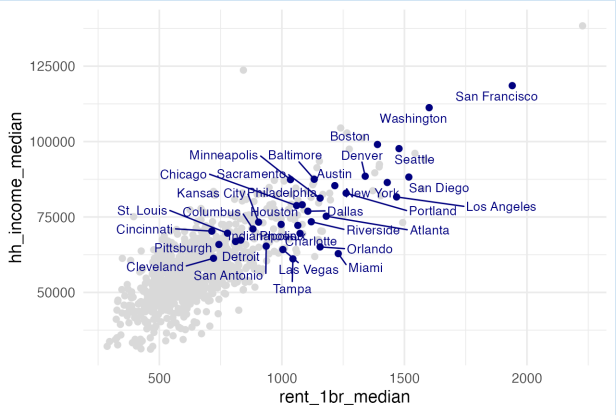
공통의 것은 위로

`aes(x, y)`는 `ggplot()`에 한 번만 쓰고 세 층이 모두 물려받는다.

층마다 다른 것은 아래로

`data`와 `color`, `size`는 층마다 따로 준다. 모두 `aes()` **밖**이다.

전체 속의 부분



읽히는 것

- 소득과 임대료는 대체로 함께 오른다.
- 다만 완전한 직선은 아니다.
- 큰 도시는 대체로 비싼 편이다.

💡 오른쪽 위 회색 점들을 보라. 크지 않은데도 큰 도시만큼 비싸다.

1 데이터 동사

2 행 고르기

3 **열 고르기와 정렬**
열 고르기
정렬

4 집단별 요약

5 새 열 만들기

6 마무리

열 고르기: select()

```
cbsa |>
  select(name, pop, lon, hh_income_median)
```

```
# A tibble: 916 x 4
  name           pop    lon hh_income_median
  <chr>        <dbl>  <dbl>            <dbl>
1 New York      20.0   -74.1            86445
2 Los Angeles   13.2  -118.             81652
3 Chicago        9.61  -88.0            78790
# i 913 more rows
```

순서도 정한다

써 준 순서대로 열이 늘어선다. 원래 자리는 무시된다.

그림에는 영향이 없다

ggplot은 어차피 aes()에 쓴 열만 쓴다. select는 주로 **출력을 보기 위해** 쓴다.

정렬: arrange()와 desc()

```
cbsa |>
  arrange(division, pop) |>
  select(name, division, pop)
```

```
1 Connersville  East N.. 0.0234
2 Scottsburg    East N.. 0.0243
3 Greensburg    East N.. 0.0265
```

```
cbsa |>
  arrange(division, desc(pop)) |>
  select(name, division, pop)
```

```
1 Chicago       East N.. 9.61
2 Detroit       East N.. 4.38
3 Cincinnati    East N.. 2.24
```

- 첫 변수로 정렬하고, 값이 같으면 두 번째 변수로 가른다. 문자는 알파벳 순이다.
- desc()로 감싼 변수는 **큰 값부터** 놓인다.

💡 정렬은 행을 버리지 않는다. 916행 그대로, 순서만 바뀐다.

상위 몇 개 뽑기: arrange + slice

```
cbsa |>
  arrange(desc(pop)) |>
  slice(1:30)
```

3강의 그 30곳

3강에서 쓴 “인구가 가장 많은 30곳”이 바로 이렇게 만들어진 자료다. `acs_cbsa.csv`가 이미 인구 순으로 저장되어 있어 `slice_head()`만으로도 같은 결과가 나왔던 것뿐이다.

관용구로 외워 두기

정렬한 다음 잘라내기. “가장 많이 등장한 단어 20개”, “가장 오래된 문서 10건”이 모두 이 두 줄이다.

순서를 바꾸면

`slice(1:30)`을 먼저 쓰면 **앞의 30행**을 자른 뒤 정렬한다. 전혀 다른 결과다.

1 데이터 동사

2 행 고르기

3 열 고르기와 정렬

4 **집단별 요약**
요약하기
집단으로 나누기
요약을 그림으로

5 새 열 만들기

6 마무리

여러 행을 한 줄로: summarize()

```
cbsa |>
  summarize(avg = mean(pop))
```

```
# A tibble: 1 x 1
    avg
  <dbl>
1 0.338
```

916행이 1행이 되었다

평균 대도시권의 인구는 33.8만 명이다. 원래 열은 모두 사라졌다. 한 줄에 담을 방법이 없기 때문이다.

쓰는 법

summarize(**새 이름** = **요약 함수**(**열 이름**)). 자주 쓰는 요약 함수는 mean, median, min, max, sum, 그리고 행의 개수를 세는 n이다.

n()만은 괄호 안이 비어 있다. 어느 열을 볼 필요 없이 행을 세기 때문이다.

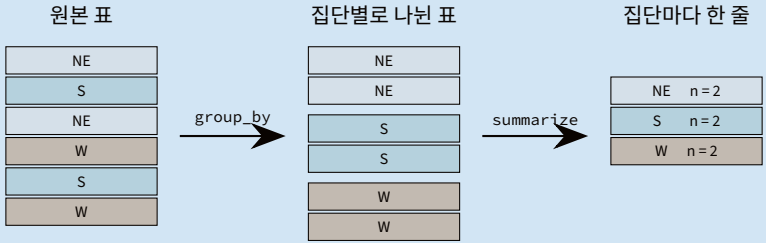
여러 요약을 한 번에

```
cbsa |>
  summarize(
    avg_hh_income_median = mean(hh_income_median),
    sum_pop = sum(pop),
    max_age_median = max(age_median),
    count = n()
  )
```

```
# A tibble: 1 x 4
  avg_hh_income_median sum_pop max_age_median count
                 <dbl>   <dbl>          <dbl> <int>
1               58466.    310.           48.8   916
```

- 콤마로 나열하면 열이 하나씩 늘어난다. 행은 여전히 하나다.
- 이름을 길게 짓는 것을 겁내지 말 것. 나중에 그림의 축 이름이 된다.

group_by(): 나누고, 적용하고, 합치기



- group_by()는 표를 눈에 보이게 바꾸지 않는다. “이제부터 집단별로 따로 하라”는 **표시**를 붙일 뿐이다.
- 그 표시를 실제로 쓰는 것은 뒤따라오는 동사다. 오늘은 summarize가 그 일을 한다.

한 줄만 더 붙였을 뿐인데

```
cbsa |>
  group_by(division) |>
  summarize(
    avg_hh_income_median = mean(hh_income_median),
    sum_pop = sum(pop),
    max_age_median = max(age_median),
    count = n()
  )
```

```
# A tibble: 9 x 5
  division   avg_hh_income_median sum_pop max_age_median
  <chr>                     <dbl>   <dbl>          <dbl>
1 East Nort..               58998.    43.3           46.6
2 East Sout..               49272.    16.7           46.4
3 Middle At..               62316.    42.1           46.9
# i 6 more rows, and 1 more variable: count <int>
```

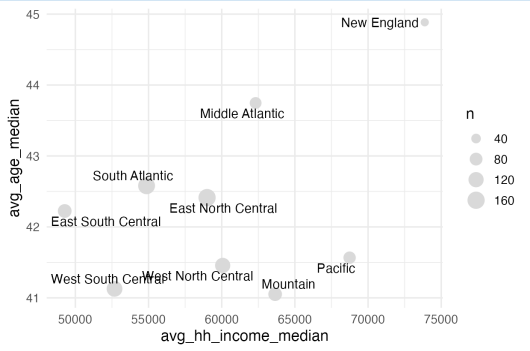
- summarize 부분은 앞 슬라이드와 글자 하나 다르지 않다. 결과는 1행에서 9행이 되었다.

요약 결과도 표다

```
cbsa |>
  group_by(division) |>
  summarize(
    avg_hh_income_median = mean(hh_income_median),
    avg_age_median = mean(age_median),
    n = n()
  ) |>
  ggplot(aes(avg_hh_income_median, avg_age_median)) +
    geom_point(aes(size = n), color = "grey85") +
    geom_text_repel(aes(label = division)) +
    scale_size_area()
```

- 요약 결과도 표이므로, 이름을 붙이지 않고 곧바로 ggplot으로 넘길 수 있다.
- summarize에서 지은 긴 이름이 그대로 축 이름이 된다.

요약 그림 읽기



점의 크기

`size = n`은 그 구분에 대도시권이 몇 곳 있는지다.
`scale_size_area()`는 점의 **넓이**를 값에 비례시킨다.

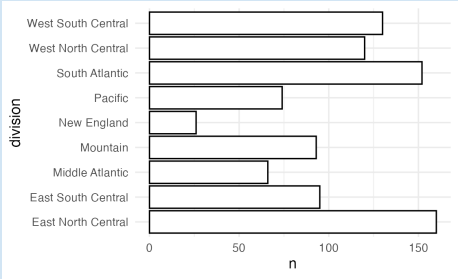
- 아홉 개의 점 뒤에는 916곳이 있다. 평균은 그 안의 편차를 지운다.
- 요약 그림으로는 **어디를 더 볼지**만 고르고, 결론은 원자료에서 확인한다.

개수 세어 막대로 그리기

```
cbsa |>
  group_by(division) |>
  summarize(n = n()) |>
  ggplot(aes(x = n, y = division)) +
    geom_col(color = "black",
             fill = "white")
```

흰 채움에 검은 테두리

무난한 출발점이다. 범주 이름이 길 때는 3강에서 배운 대로 가로 막대가 낫다.



또 알파벳 순이다. 이 문제는 다음 절에서 끝낸다.

1 데이터 동사

2 행 고르기

3 열 고르기와 정렬

4 집단별 요약

5 **새 열 만들기**
mutate
범주의 순서

6 마무리

새 열 만들기: mutate()

```
cbsa |>
  mutate(pop_100k = pop * 10) |>
  select(name, pop, pop_100k)
```

```
# A tibble: 916 x 3
  name            pop pop_100k
  <chr>         <dbl>    <dbl>
1 New York      20.0      200.
2 Los Angeles   13.2      132.
3 Chicago        9.61      96.1
# i 913 more rows
```

행은 그대로

summarize와 달리 행을 접지 않는다.
916행 그대로, 열만 하나 늘었다.

새 열은 맨 뒤에

여기서는 select로 앞으로 끌어왔다.
기존 열 이름을 쓰면 그 열을 **덮어쓴다**
(자리는 그대로).

조건에 따라 값 정하기: if_else()

```
cbsa |>
  mutate(region_size = if_else(pop > 7, "large", "small")) |>
  select(name, pop, region_size)
```

```
# A tibble: 916 x 3
  name            pop region_size
  <chr>         <dbl> <chr>
1 New York      20.0  large
2 Los Angeles   13.2  large
3 Chicago        9.61 large
4 Dallas         7.54 large
5 Houston        7.05 large
6 Washington     6.33 small
# i 910 more rows
```

- if_else()는 세 부분이다. **참/거짓 문장, 참일 때 값, 거짓일 때 값.**
- 이렇게 만든 범주는 여느 문자열 열과 똑같다. 색에 이어도 되고, 묶어서 요약해도 된다.

범주를 여러 개로

```
cbsa |>
  mutate(rs = "default") |>   # 새 열 rs를 만든다
  mutate(rs = if_else(pop > 7, "huge", rs)) |>
  mutate(rs = if_else(between(pop, 2, 7), "large", rs)) |>
  mutate(rs = if_else(between(pop, 1, 2), "medium", rs)) |>
  mutate(rs = if_else(between(pop, 0.3, 1), "small", rs)) |>
  mutate(rs = if_else(pop < 0.3, "tiny", rs)) |>
  group_by(rs) |>
  summarize(pop_sum = sum(pop), n = n())
```

- 첫 줄의 `mutate(rs = "default")`가 앞 슬라이드의 region_size에 해당하는 새 열을 만든다. 이름이 길어 rs로 줄였을 뿐이다.
- 다섯 범주를 한 번에 만들려 하지 않고, **기본값을 깔고 하나씩 덮어쓴다**.

❗ "거짓이면 자기 자신"(rs)은 **바꾸지 말라**는 뜻이다.

왜 "default"부터 넣는가

```
# A tibble: 5 x 3
  rs      pop_sum     n
  <chr>     <dbl> <int>
1 huge       57.4     5
2 large     101.     29
3 medium     28.5    21
4 small      60.1   108
5 tiny       62.6   753
```

➔ 다섯 곳에 5,740만 명, 753곳에 6,260만 명. 인구 분포의 비대칭이 한 줄짜리 표에 담겼다.

한 줄 아낄 수 있는데도 굳이 넣는 이유

1. **읽기 쉽다**. "tiny는 나머지"가 아니라 다섯 조건이 모두 눈에 보인다.
2. **검산이 된다**. `default`가 하나라도 남아 있으면 조건을 빠뜨렸다는 뜻이다.
3. **결측에 안전하다**. 값이 비어 있는 행이 조용히 엉뚱한 범주로 들어가는 일을 막는다.

알파벳 순을 벗어나기: 요인(factor)

왜 알파벳 순인가

문자열 열(<chr>)에는 순서가 없다. 그래서 ggplot은 알파벳 순으로 늘어놓는다.

요인(<fct>)

범주에 **정해진 순서**(수준, level)를 붙인 자료형이다. 그 순서대로 그림과 범례가 놓인다.

쓰는 자리

⚠ 자료를 요인으로 **저장해 두지는** 않는다. 엉뚱한 오류를 일으키기 쉽다.

✓ 그림을 그리기 직전 `mutate` 안에서 잠깐 만든다.

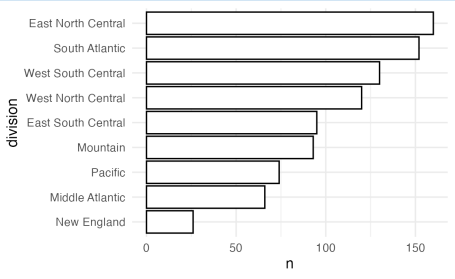
`fct_inorder()`

자료에 나타난 순서 그대로 수준을 매긴다. 그래서 `arrange()`와 짝을 이룬다.

순서를 고친 막대그래프

```
cbsa |>
  group_by(division) |>
  summarize(n = n()) |>
  arrange(n) |>
  mutate(division =
           fct_inorder(division)) |>
  ggplot(aes(x = n, y = division)) +
    geom_col(color = "black",
             fill = "white")
```

✓ 3강에서 미뤄 둔 문제를 풀었다. 고친 것은 그림이 아니라 **자료**다.



💡 `fct_relevel()`은 특정 범주를 맨 앞으로, `fct_lump_n()`은 작은 범주들을 "기타"로 묶는다.

1 데이터 동사

2 행 고르기

3 열 고르기와 정렬

4 집단별 요약

5 새 열 만들기

6 **마무리**

일곱 동사 한눈에

동사	보기	행 / 열
`slice()`	`slice(1:30)`	행↓
`filter()`	`filter(pop > 5)`	행↓
`select()`	`select(name, pop)`	열↓
`arrange()`	`arrange(desc(pop))`	그대로 (순서만)
`mutate()`	`mutate(x = pop * 10)`	열↑
`group_by()`	`group_by(division)`	그대로 (표시만)
`summarize()`	`summarize(n = n())`	행↓↓

`filter`, `select`, `group_by`라는 이름은 데이터베이스 언어인 SQL에서 왔고, Python의 pandas에도 같은 개념이 있다. 오늘 배운 것은 R 밖에서도 쓴다.

오늘 배운 것

행과 열

- `slice()`, `filter()`
- `between()`, `%in%`
- `select()`, `arrange()`, `desc()`

요약

- `summarize()`와 `n()`
- `group_by()` — 나누고, 적용하고, 합치기

새 열

- `mutate()`, `if_else()`
- `fct_inorder()`로 범주의 순서 정하기

그림과의 연결

- 층마다 `data =`로 다른 표 쓰기
- 요약 결과를 그대로 `ggplot`에 넘기기

확인 질문

스스로 답해 보기

1. `cbsa |> filter(pop > 5)`를 실행한 뒤 cbsa는 몇 행인가?
2. `filter(quad = "NE")`라고 쓰면 왜 안 되는가?
3. `summarize()`는 왜 원래의 열들을 남기지 못하는가?
4. `group_by(division)`만 실행하면 화면에 무엇이 달라지는가?
5. 막대의 순서를 바꾸려고 ggplot 쪽 코드를 고치는 것은 왜 잘못된 접근인가?

💡 답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간에 배울 것

- 제5강 탐색적 자료 분석 (3): 여러 표 다루기 [교재 4장]
  - 정돈된 데이터(tidy data)란 무엇인가
  - 흩어진 표를 결합하기(join), 표의 모양을 바꾸기(pivot)

다음 시간까지 할 일

1. 오늘의 동사 가운데 셋을 골라 파이프로 이어 붙인 코드를 하나 써 보기
2. 3강에서 그린 그림 하나를 골라, 자료를 먼저 손질한 뒤 다시 그려 보기

[숙제03] 데이터 요약하기 (3점)

- 파일명: `hw03-2026-00000.Rmd` (`2026-00000`은 자신의 학번)
- 제출 기한: **2026-09-21 09:00** (월요일 오전 9시) eTL
- 오늘 배운 내용으로 충분히 풀 수 있다. 자세한 안내는 eTL 공지 참조.
- Knit이 오류 없이 되는지 반드시 확인하고 제출할 것!

참고 자료

- `dplyr` 치트시트 — `https://rstudio.github.io/cheatsheets/data-transformation.pdf`
- `forcats` 치트시트 — `https://rstudio.github.io/cheatsheets/factors.pdf`